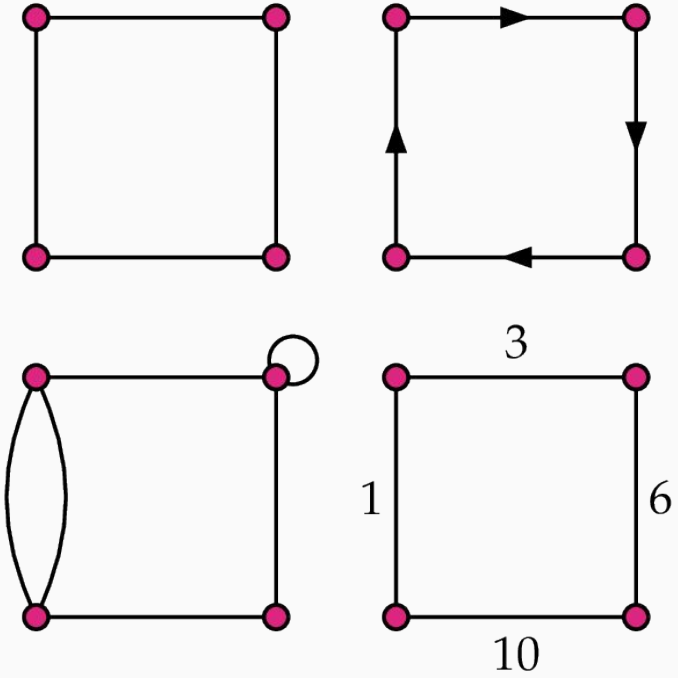
Graph Representations

Prof. Timothy Sun

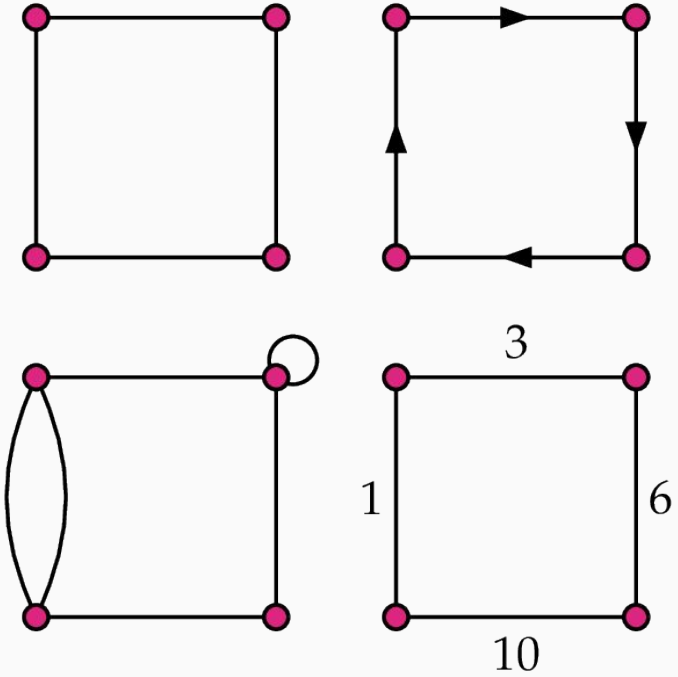
Graphs

A graph ***G*** consists of vertices ***V*** and edges ***E***.



Graphs

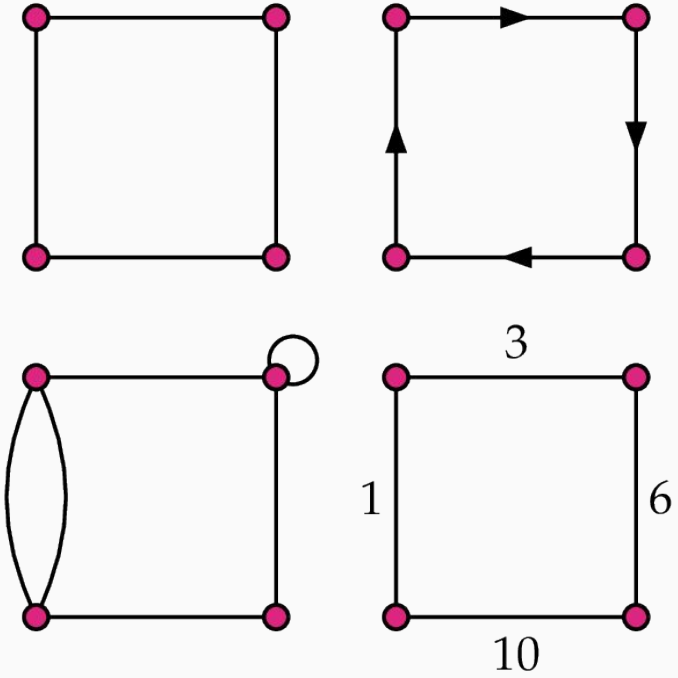
A graph ***G*** consists of vertices ***V*** and ***edges E***.



Graphs

Edges can be:

- directed
- weighted
- non-simple

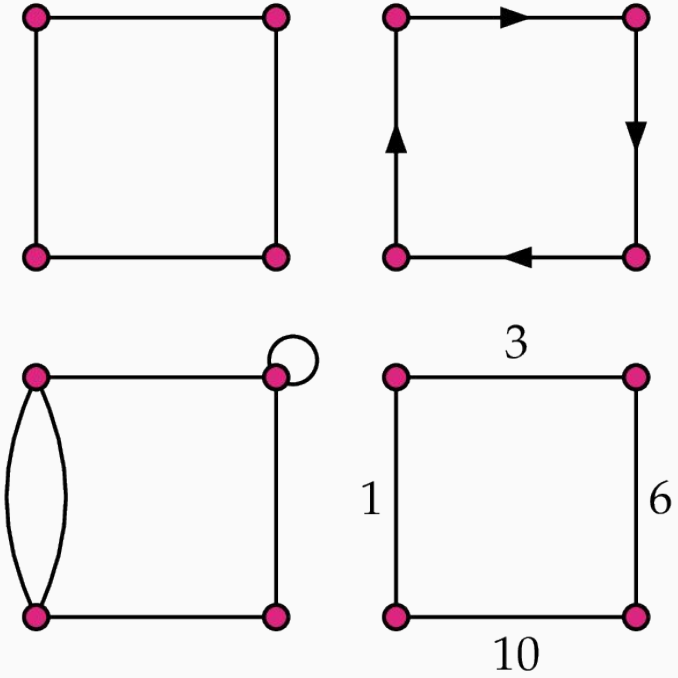


Graphs

Edges can be:

- directed
- weighted
- non-simple

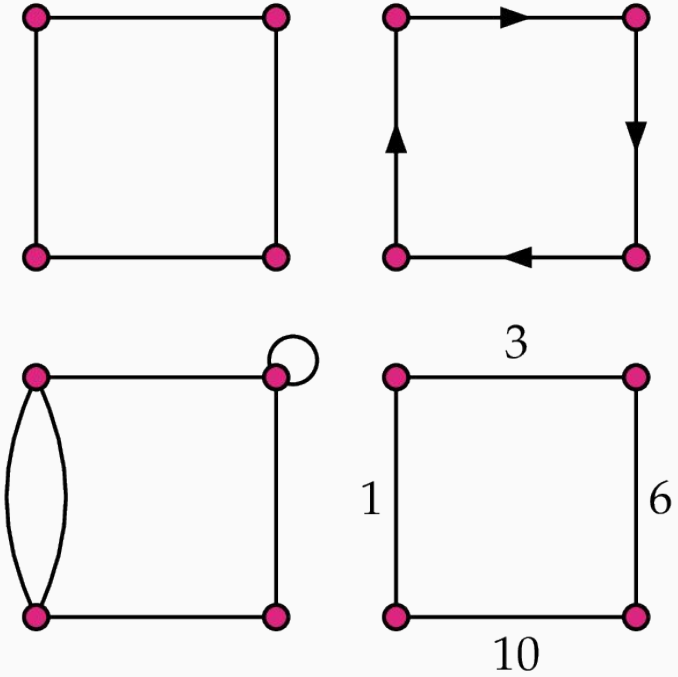
Edges capture **pairwise** relationships.



Graph representations

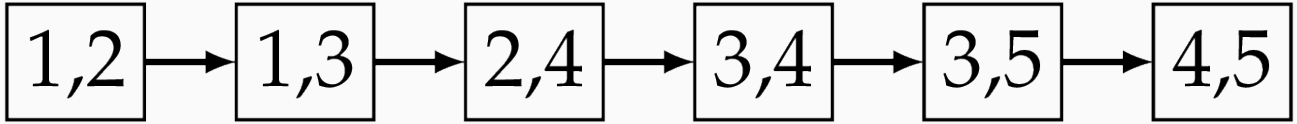
Two fundamental operations:

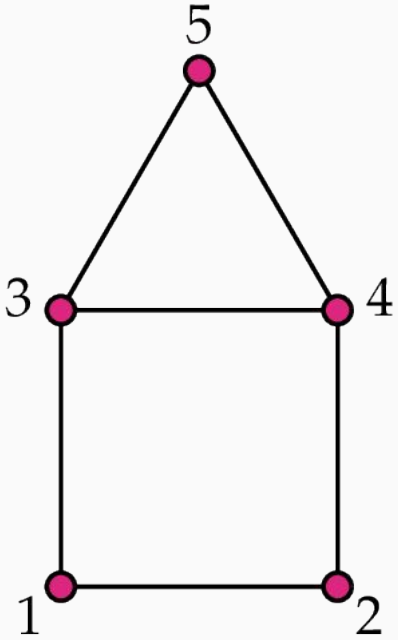
- “Is there an edge between ***u*** and ***v***?”
- “What are all the edges leaving ***v***?”



(Edge list)

Store all edges in a list:

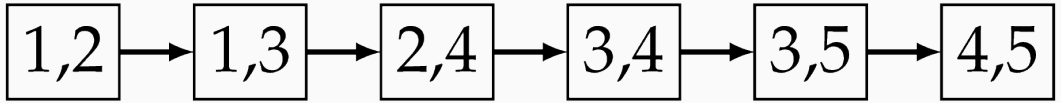


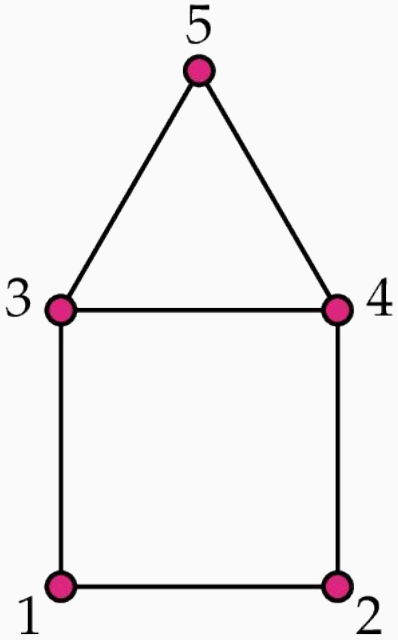


(Edge list)

Store all edges in a list:

- "Is there an edge between ***u*** and ***v***?"
- "What are all the edges leaving ***v***?"

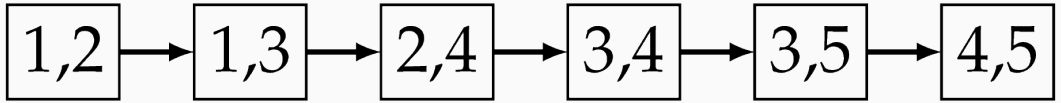


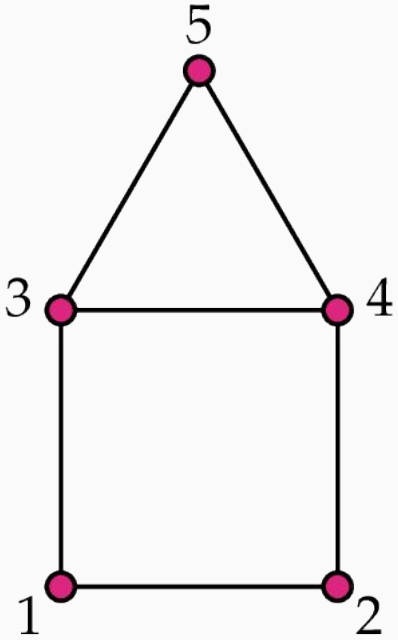


(Edge list)

Store all edges in a list:

- "Is there an edge between ***u*** and ***v***?" $O(|E|)$
- "What are all the edges leaving ***v***?" $\Theta(|E|)$

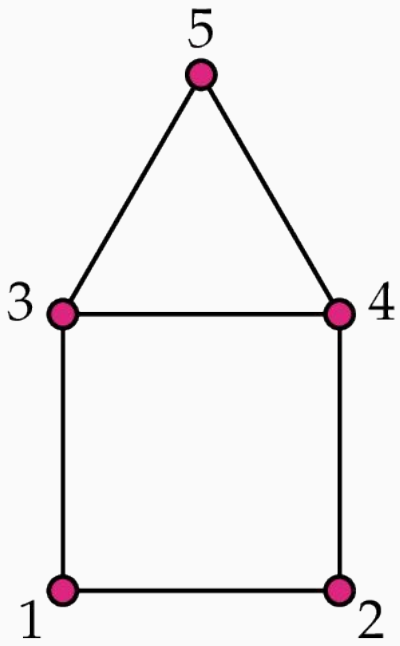




Adjacency matrix

Store all edge information in a matrix:

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	0
3	1	0	0	1	1
4	0	1	1	0	1
5	0	0	1	1	0

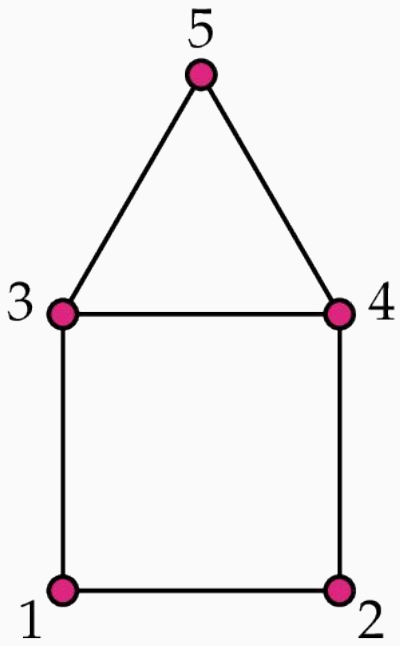


Adjacency matrix

Store all edge information in a matrix.

- "Is there an edge between ***u*** and ***v***?"
- "What are all the edges leaving ***v***?"

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	0
3	1	0	0	1	1
4	0	1	1	0	1
5	0	0	1	1	0

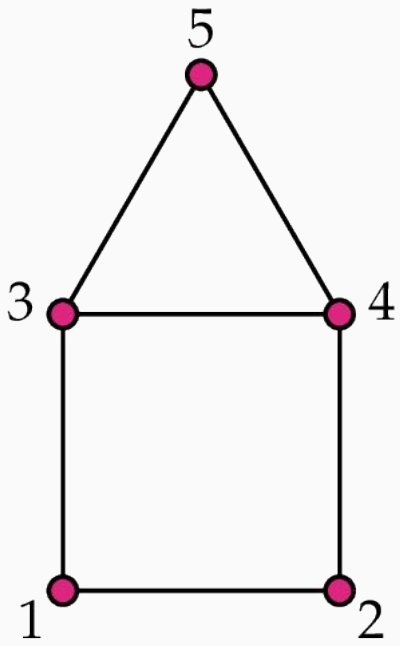


Adjacency matrix

Store all edge information in a matrix.

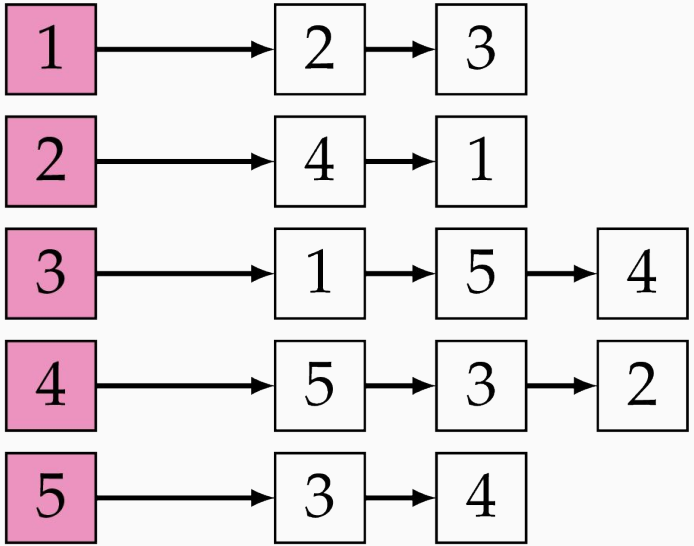
- "Is there an edge between ***u*** and ***v***?" $\Theta(1)$
- "What are all the edges leaving ***v***?" $\Theta(|\boldsymbol{V}|)$

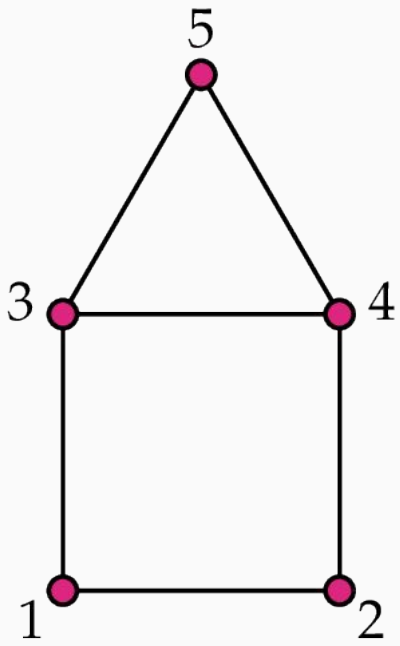
	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	0
3	1	0	0	1	1
4	0	1	1	0	1
5	0	0	1	1	0



Adjacency list

Each vertex has a list of outgoing edges/neighbors:

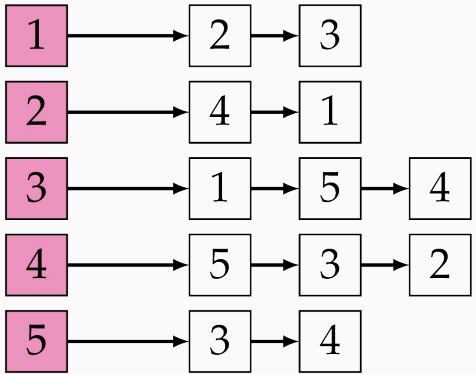


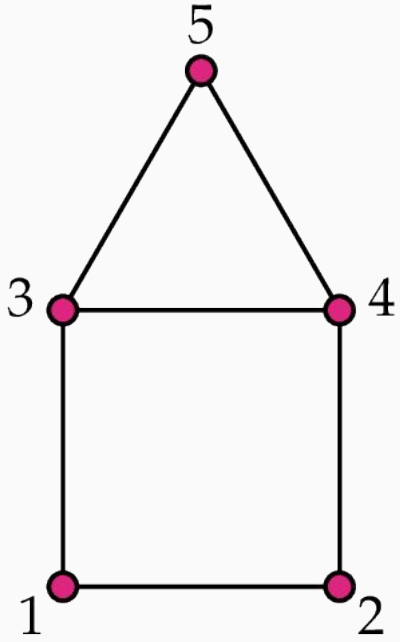


Adjacency list

Each vertex has a list of outgoing edges/neighbors:

- "Is there an edge between ***u*** and ***v***?"
- "What are all the edges leaving ***v***?"



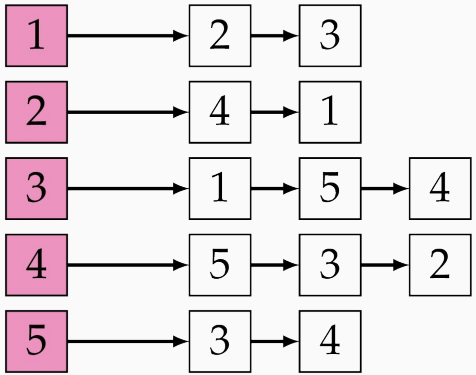


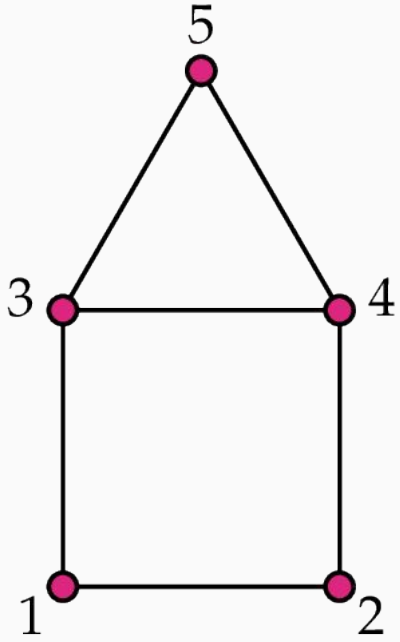
Adjacency list

Each vertex has a list of outgoing edges/neighbors:

- "Is there an edge between ***u*** and ***v***?"
- "What are all the edges leaving ***v***?"

d(***v***) is the ***degree*** of ***v***.



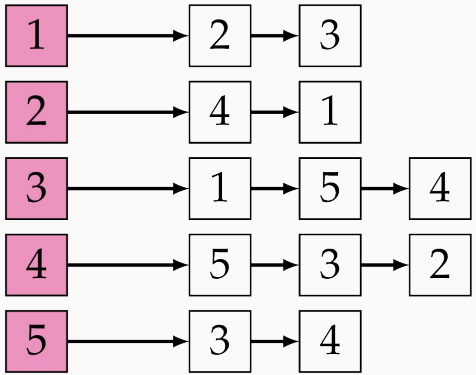


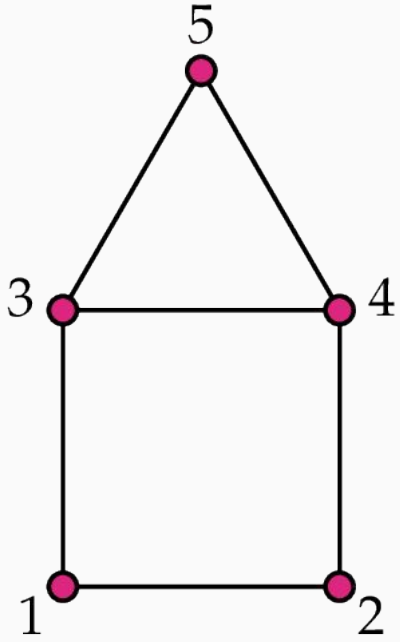
Adjacency list

Each vertex has a list of outgoing edges/neighbors:

- "Is there an edge between ***u*** and ***v***?" O(min(d(***u***), d(***v***)))
- "What are all the edges leaving ***v***?" Θ(d(***v***))

d(***v***) is the ***degree*** of ***v***.





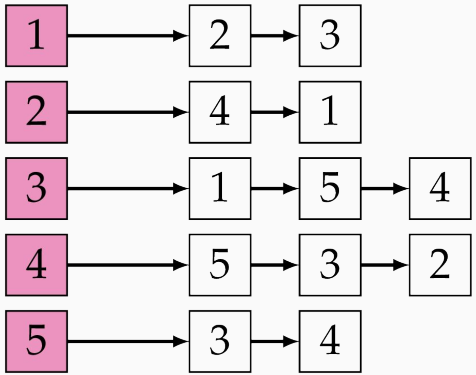
Adjacency list

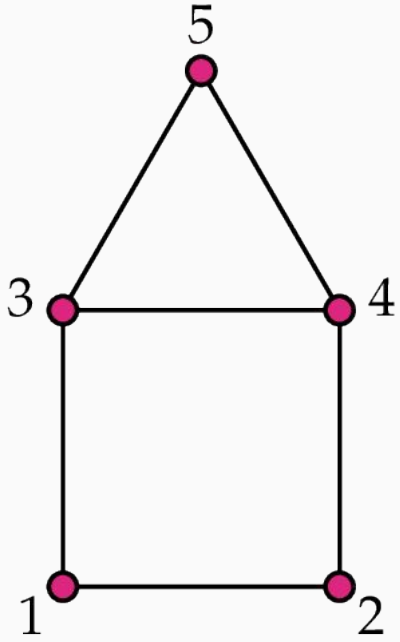
Each vertex has a list of outgoing edges/neighbors:

- "Is there an edge between ***u*** and ***v***?" O(min(d(***u***), d(***v***)))
- "What are all the edges leaving ***v***?" Θ(d(***v***))

d(***v***) is the ***degree*** of ***v***.

(for directed graphs, out/indegree)

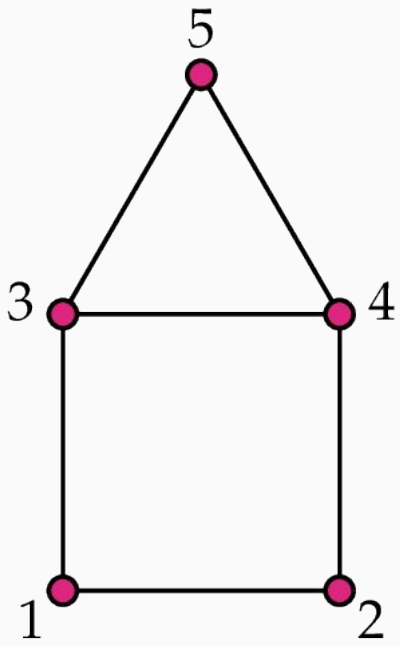




Adjacency map

Each vertex has a (hash)**map** of outgoing edges:

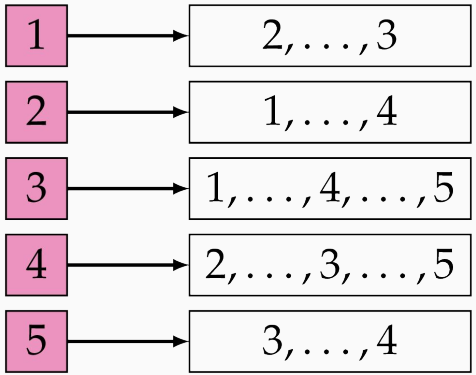
1	2, . . . , 3
2	1, . . . , 4
3	1, . . . , 4, . . . , 5
4	2, . . . , 3, . . . , 5
5	3, . . . , 4

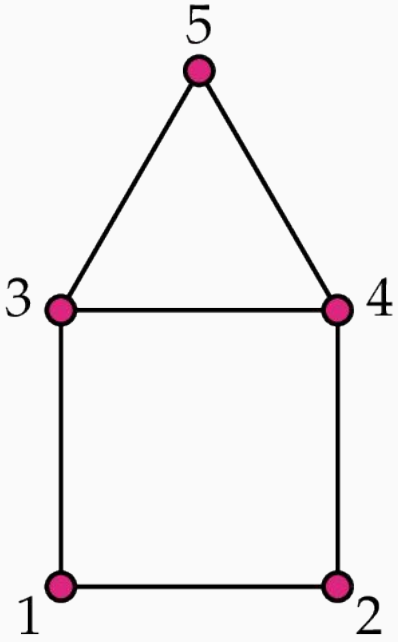


Adjacency map

Each vertex has a (hash)**map** of outgoing edges:

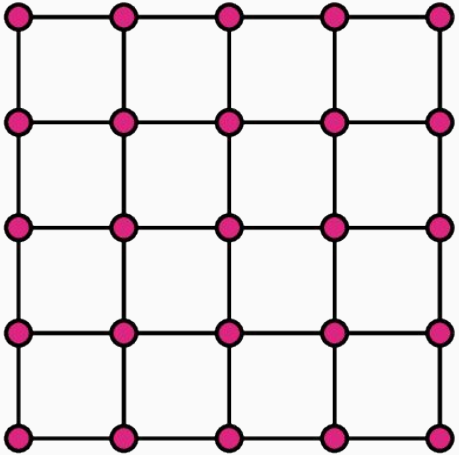
- "Is there an edge between ***u*** and ***v***?" Θ(1)-ish...
- "What are all the edges leaving ***v***?" Θ(d(***v***))





Which is the best?

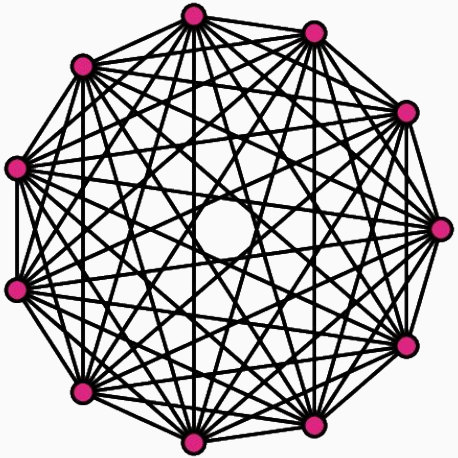
Adjacency **lists** seem better when graphs are **sparse**.

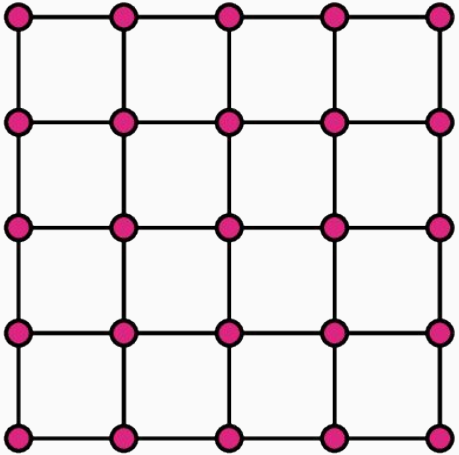


Which is the best?

Adjacency **lists** seem better when graphs are **sparse**.

Adjacency **matrices** seem better when graphs are **dense**.





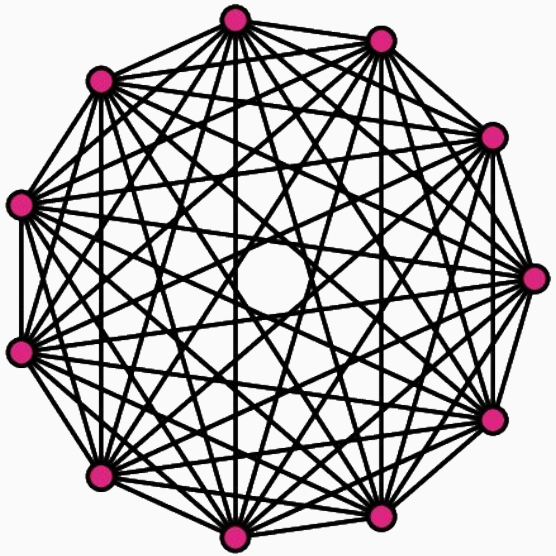
Which is the best?

Don't **adjacency maps** have the best of both worlds?

	List	Matrix	Map
is_edge(***u***,***v***)	min(d(***u***), d(***v***))	1	1 ("expected")
neighbors(***v***)	d(***v***)	|***V***|	d(***v***)

Which is the best?

- Matrices better on very dense graphs.



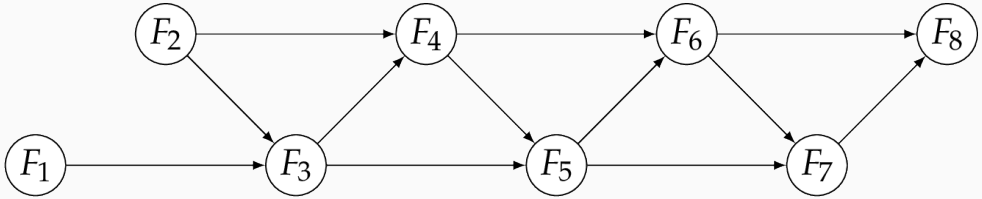
Which is the best?

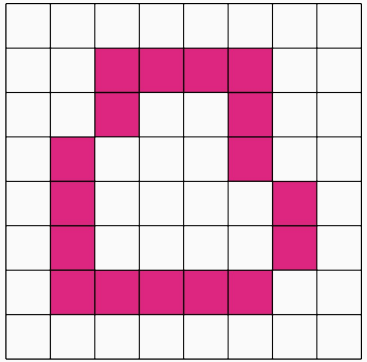
- Matrices better on very dense graphs.
- Many algorithms (e.g. DFS) don't use the edge query operation.

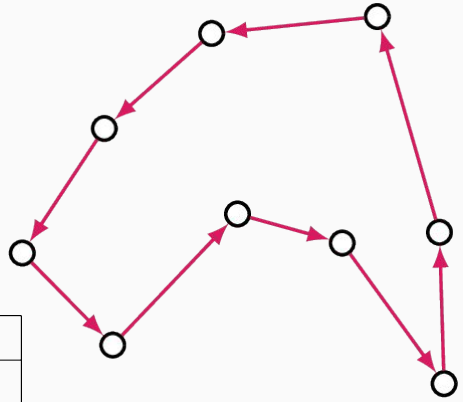
```
DFS(v):
    if v is unmarked
        mark v
        for each edge (v, w)
            DFS(w)
```

Which is the best?

- Matrices better on very dense graphs.
- Many algorithms (e.g. DFS) don't use the edge query operation.
- Graphs are sometimes **implicit**.







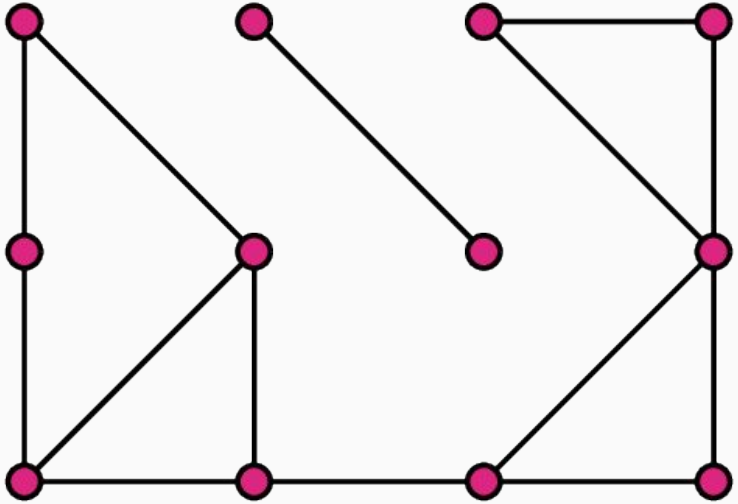
Which is the best?

- Matrices better on very dense graphs.
- Many algorithms (e.g. DFS) don't use the edge query operation.
- Graphs are sometimes **implicit**.

We'll only use **adjacency lists** from here on.

Connectivity

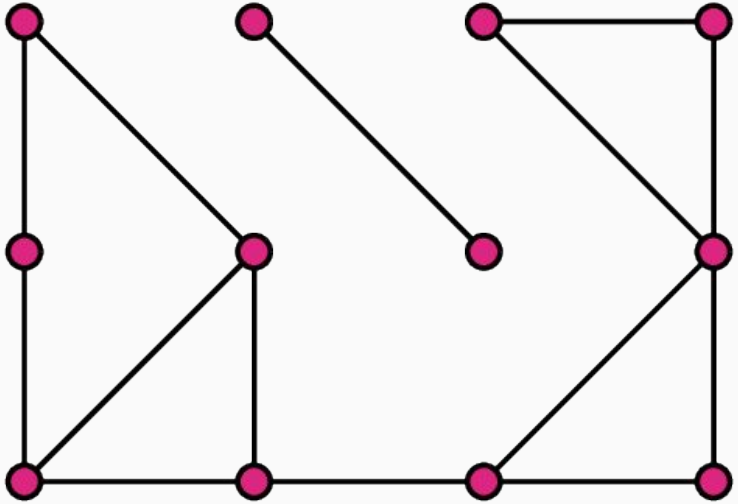
Is this graph connected?



Connectivity

Is this graph connected?

No, there are two “islands”

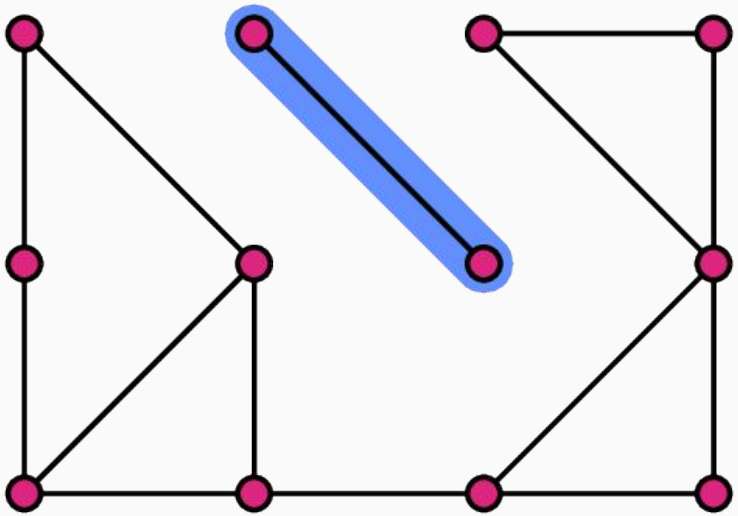


Connectivity

Is this graph connected?

No, there are two “islands”

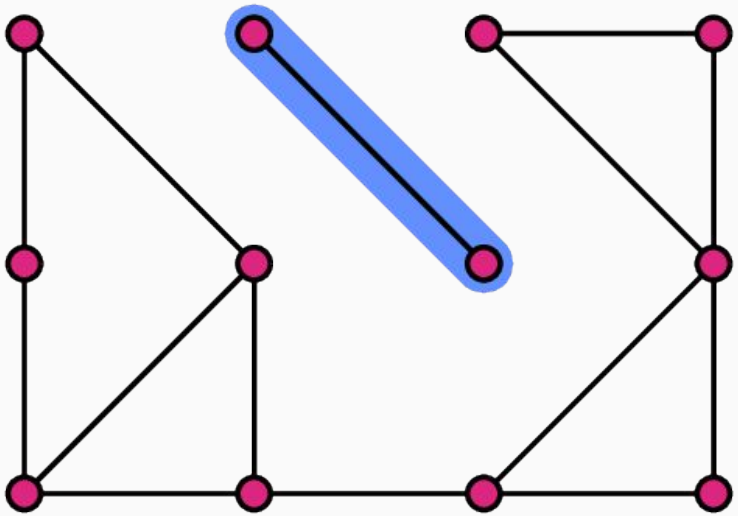
One island is not “reachable” from the other.



Connectivity

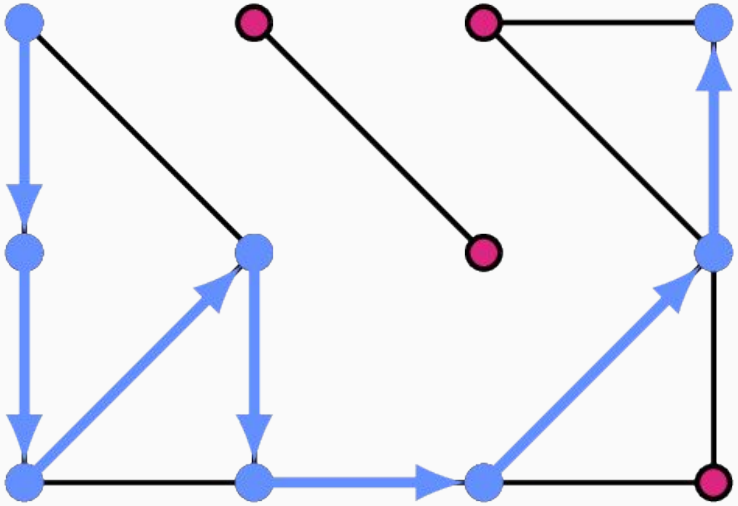
A graph is ***connected*** if you can walk from any vertex to any other vertex.

A (***connected***) ***component*** of an undirected graph is a maximal connected subgraph.



Reachability

A ***walk*** starts at one vertex and moves along a sequence of edges to another vertex.

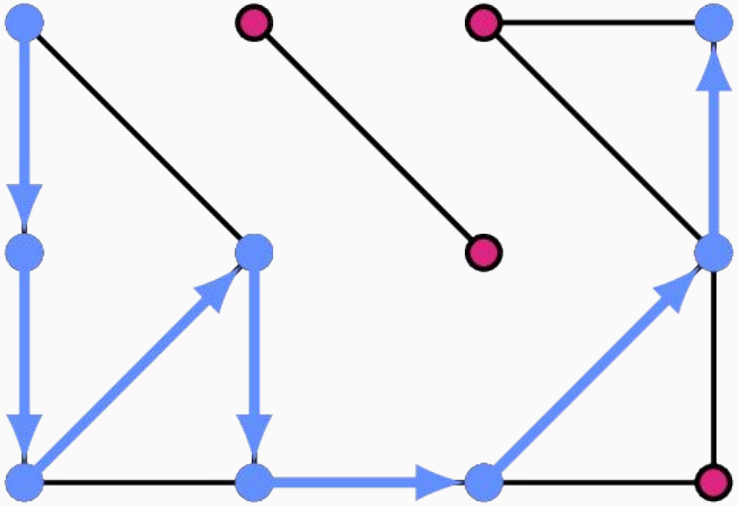


Reachability

A ***walk*** starts at one vertex and moves along a sequence of edges to another vertex.

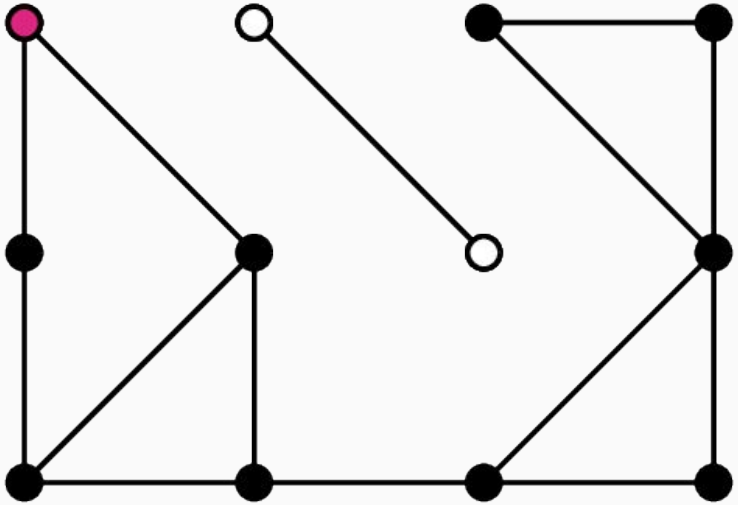
A ***path*** is a walk with no repetitions.

Given a specific vertex ***v***, which vertices are **reachable** from ***v*** by a walk/path?



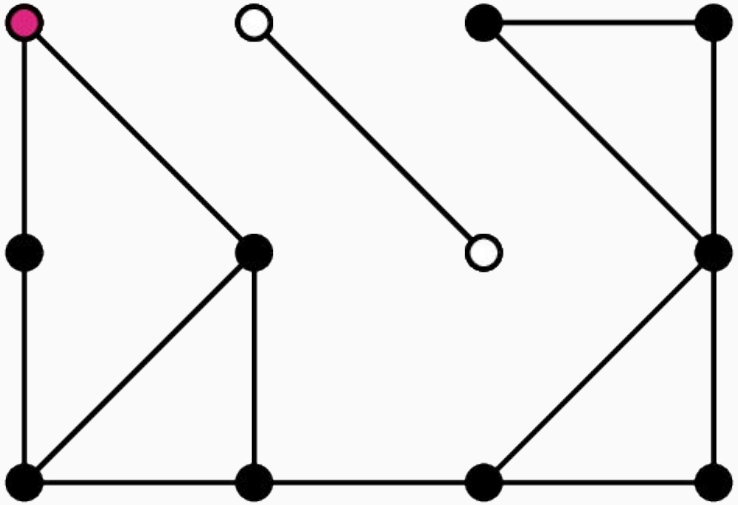
Reachability

Given a specific vertex ***v***, which vertices are **reachable** from ***v*** by a walk/path?



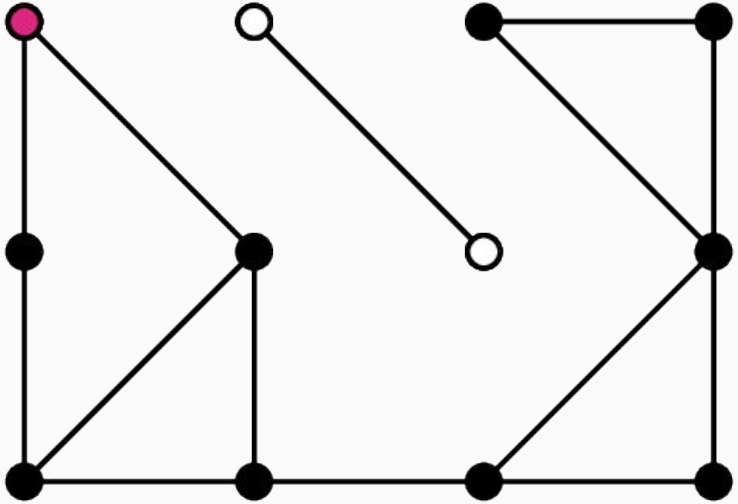
“Something-first” search

```
SomethingFirst(G, v):
    put v into bag
    while bag is non-empty
        take s from bag
        if s is unmarked
            mark s
            for each edge (s, w) ∈ E(G)
                put w into bag
```



“Something-first” search

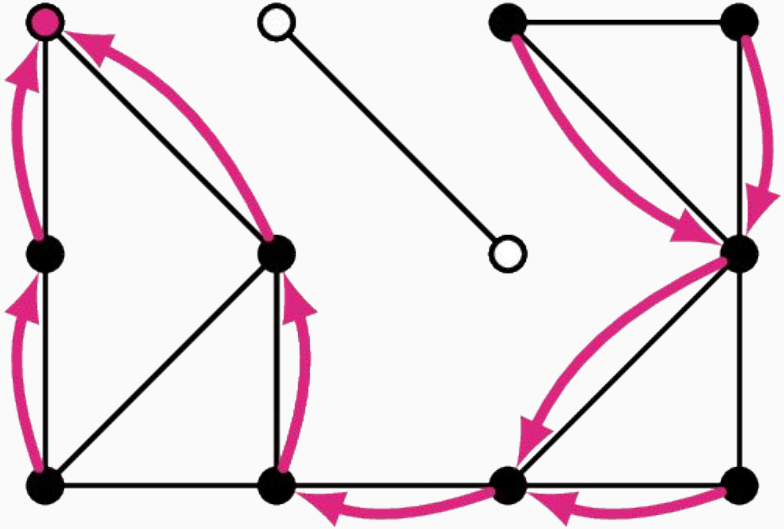
```
SomethingFirst(G, v):
    put v into bag
    while bag is non-empty
        take s from bag
        if s is unmarked
            mark s
            for each edge (s, w) ∈ E(G)
                put w into bag
```



Proof sketch: induct on BFS layers

“Something-first” search

```
SomethingFirst(G, v):
    put (null, v) into bag
    while bag is non-empty
        take (p, s) from bag
        if s is unmarked
            mark s
            parent(s) := p
            for each edge (s, w) ∈ E(G)
                put (s, w) into bag
```



“Something-first” search

What’s the runtime?

```
SomethingFirst(G, v):
    put (null, v) into bag
    while bag is non-empty
        take (p, s) from bag
        if s is unmarked
            mark s
            parent(s) := p
            for each edge (s, w) ∈ E(G)
                put (s, w) into bag
```

“Something-first” search

What’s the runtime?

- **Depends on the bag**

```
SomethingFirst(G, v):
    put (null, v) into bag
    while bag is non-empty
        take (p, s) from bag
        if s is unmarked
            mark s
            parent(s) := p
            for each edge (s, w) ∈ E(G)
                put (s, w) into bag
```

“Something-first” search

What’s the runtime?

- **Depends on the bag:** if insert and remove take O(***T***) time, then total runtime is O(***E’T***), where ***E’*** is the number of edges in ***v***’s component

```
SomethingFirst(G, v):
    put (null, v) into bag
    while bag is non-empty
        take (p, s) from bag
        if s is unmarked
            mark s
            parent(s) := p
            for each edge (s, w) ∈ E(G)
                put (s, w) into bag
```

“Something-first” search

What’s the runtime?

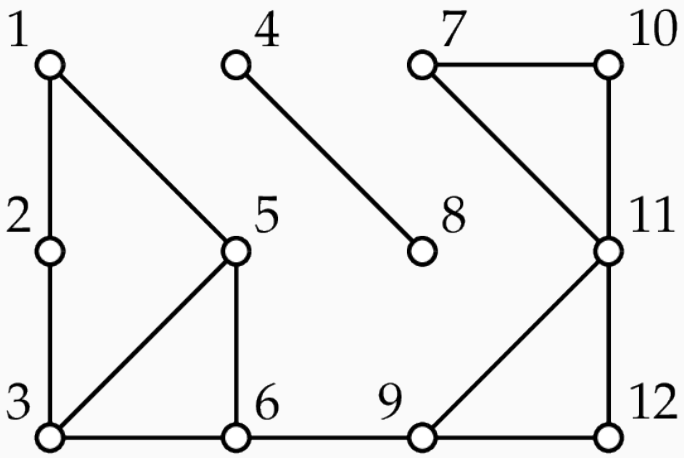
- **Depends on the bag:** if insert and remove take O(***T***) time, then total runtime is O(***E’T***), where ***E’*** is the number of edges in ***v***’s component

When “bag” is a stack or queue, ***T*** = O(1).

```
SomethingFirst(G, v):
    put (null, v) into bag
    while bag is non-empty
        take (p, s) from bag
        if s is unmarked
            mark s
            parent(s) := p
            for each edge (s, w) ∈ E(G)
                put (s, w) into bag
```

Connected components

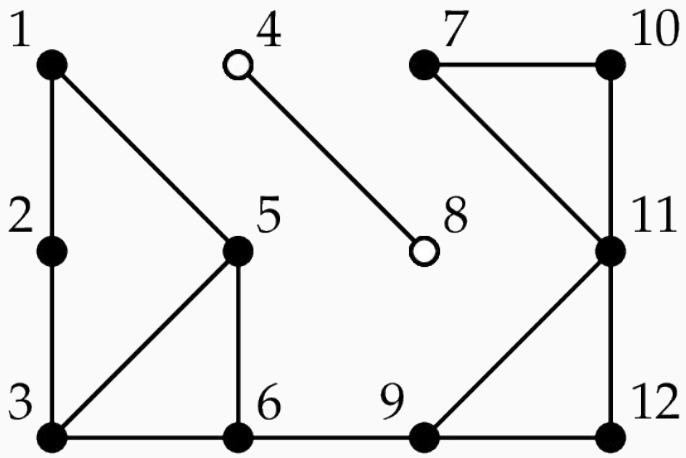
```
CountComp(G):
    count := 0
    for each v ∈ V(G)
        if v is unmarked
            count := count + 1
            SomethingFirst(G, v)
    return count
```



Try doing a “something-first” search on every vertex.

Connected components

```
CountComp(G):
    count := 0
    for each v ∈ V(G)
        if v is unmarked
            count := count + 1
            SomethingFirst(G, v)
    return count
```



Try doing a “something-first” search on every vertex.

Connected components

$\textsc{CountComp}(G)$:

$\quad count := 0$

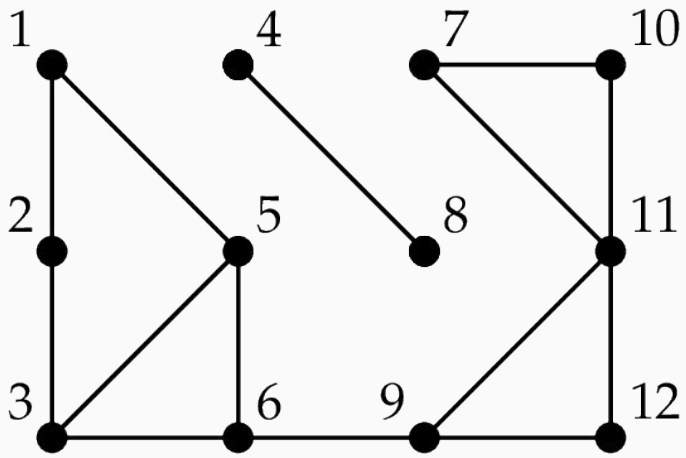
$\quad$ **for** each $v \in V(G)$

$\quad\quad$ **if** v is unmarked

$\quad\quad\quad count := count + 1$

$\quad\quad\quad \textsc{SomethingFirst}(G, v)$

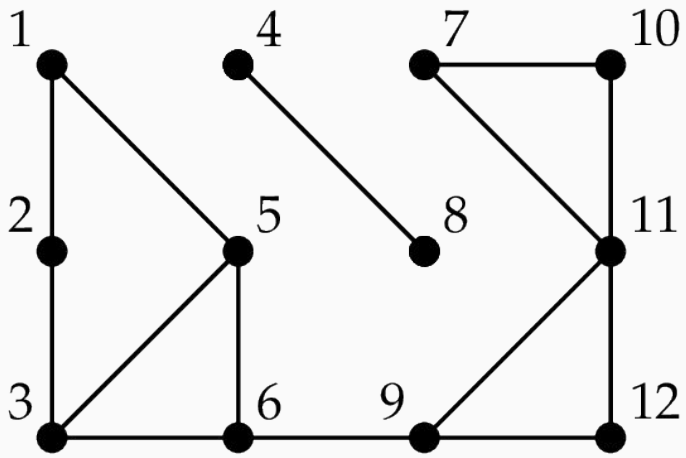
$\quad$ **return** $count$



Try doing a “something-first” search on every vertex.

Connected components

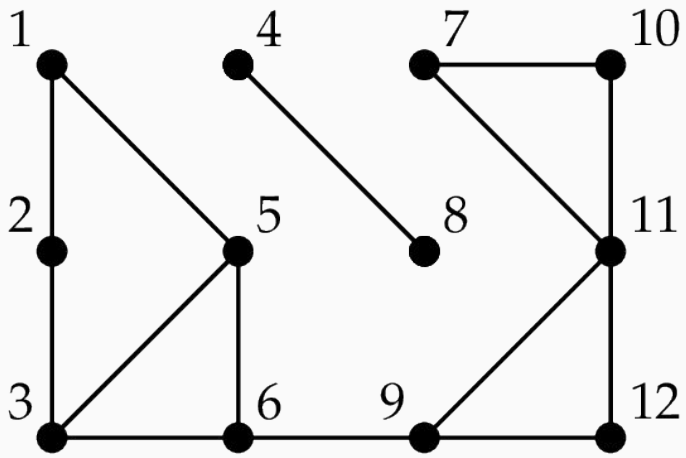
```
CountComp(G):
    count := 0
    for each v ∈ V(G)
        if v is unmarked
            count := count + 1
            SomethingFirst(G, v)
    return count
```



Runtime: O(|***V***| + |***E***|)

Connected components

```
CountComp(G):
    count := 0
    for each v ∈ V(G)
        if v is unmarked
            count := count + 1
            SomethingFirst(G, v)
    return count
```



Runtime: O(|***V***| + |***E***|) ← linear time

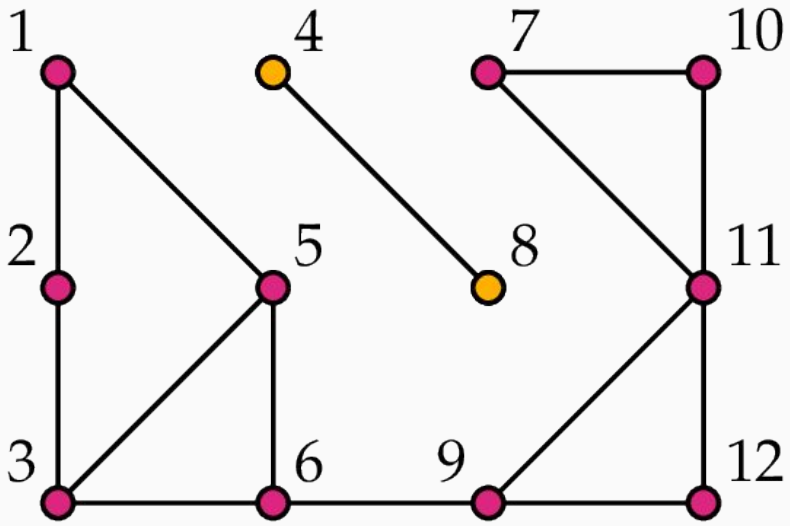
Connected components

```
CountComp(G):
    count := 0
    for each v ∈ V(G)
        if v is unmarked
            count := count + 1
            SomethingFirst(G, v, count)
    return count
```

```
SomethingFirst(G, v, i):
    put v into bag
    while bag is non-empty
        take s from bag
        if s is unmarked
            mark s with i
            for each edge (s, w) ∈ E(G)
                put w into bag
```

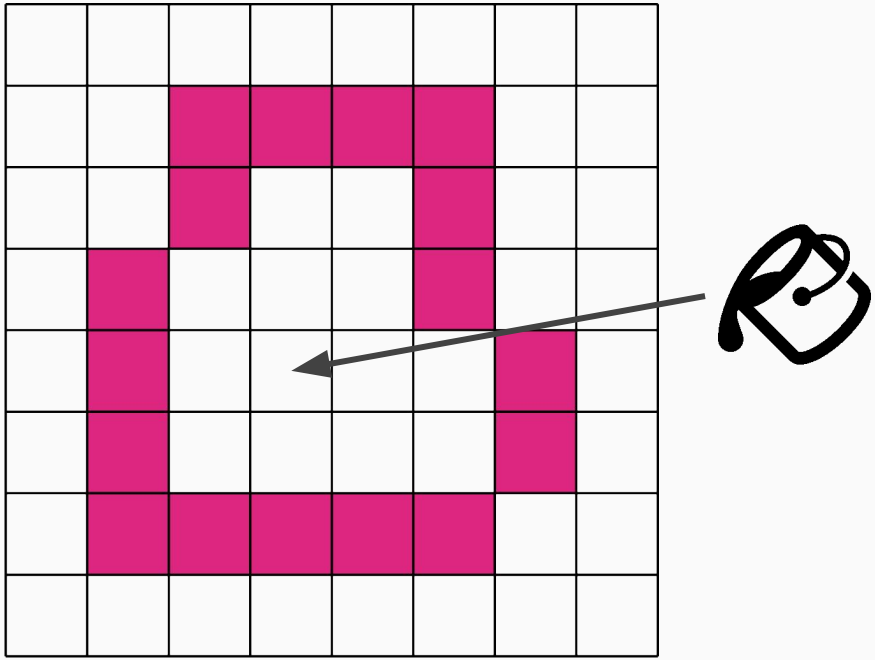
Can extend slightly to record component labels

Connected components

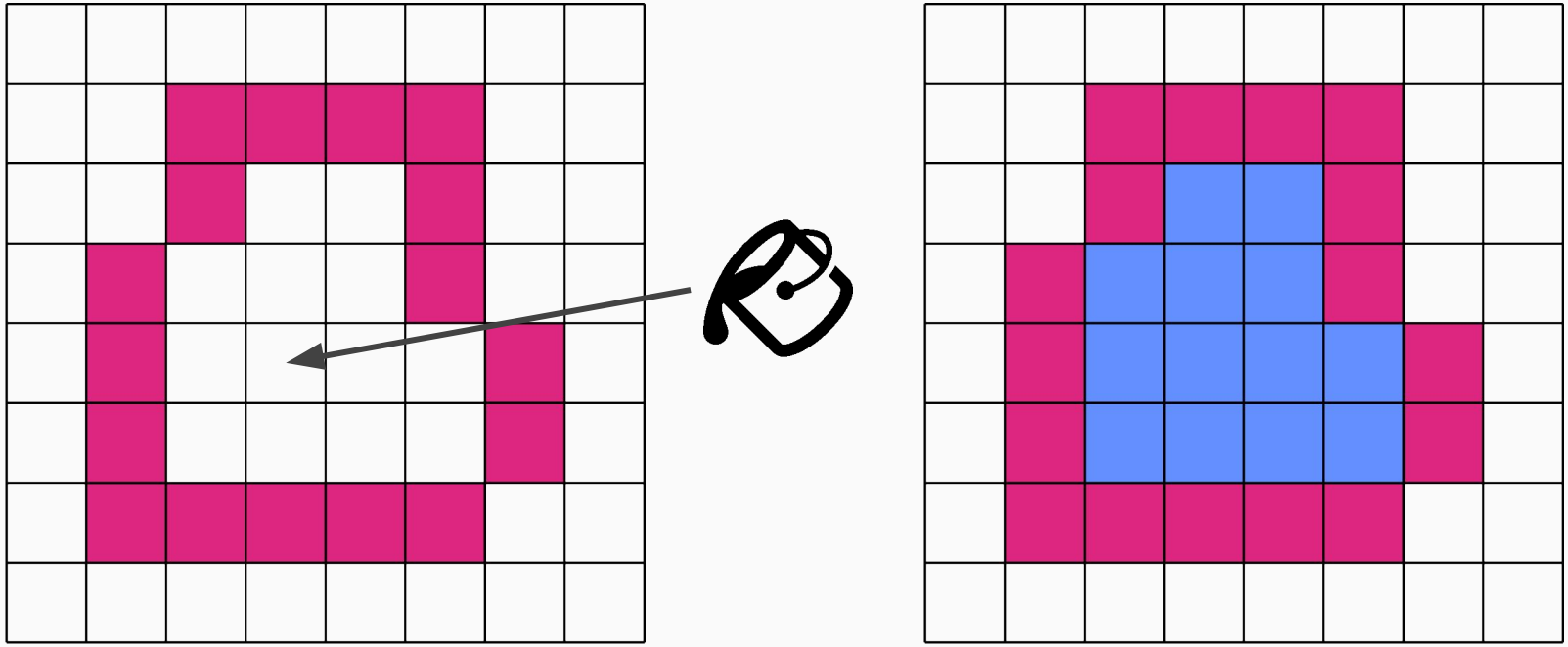


Can extend slightly to record component labels

“Fill” tool in Paint



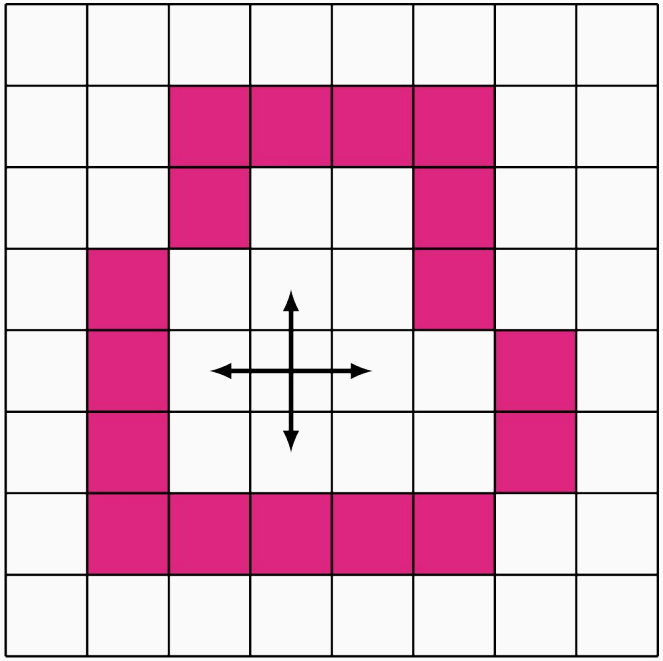
“Fill” tool in Paint



“Fill” tool in Paint

The fill rule:

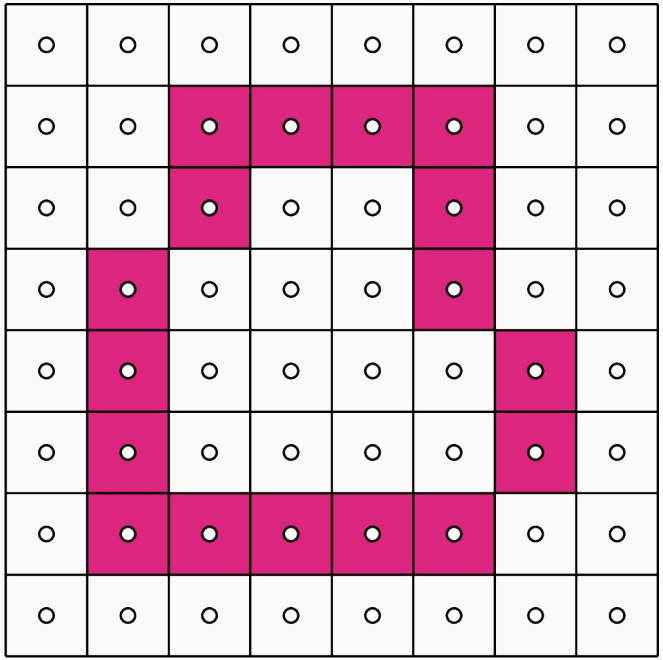
- Color orthogonal neighbors if they have the same color.
- Repeat until no more pixels remain.



“Fill” tool in Paint

“Pretend” like you have a graph.

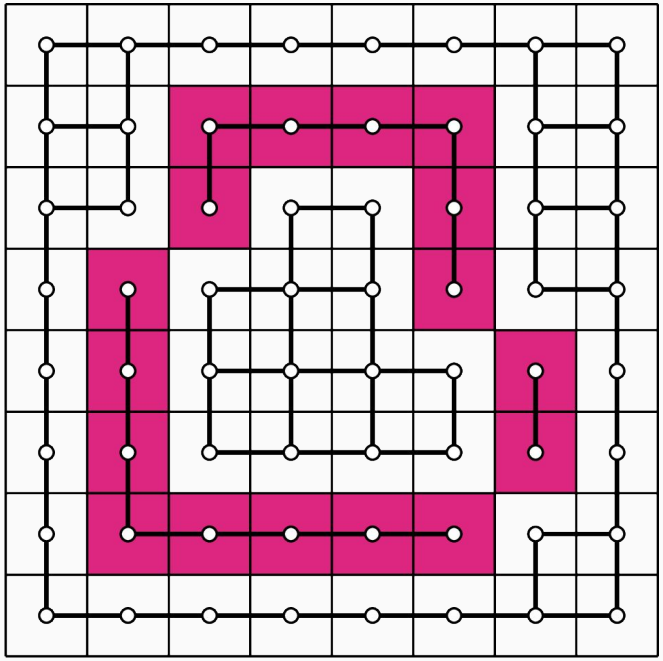
- Vertices are pixels



“Fill” tool in Paint

“Pretend” like you have a graph.

- Vertices are pixels
- Edge if neighbors are same color

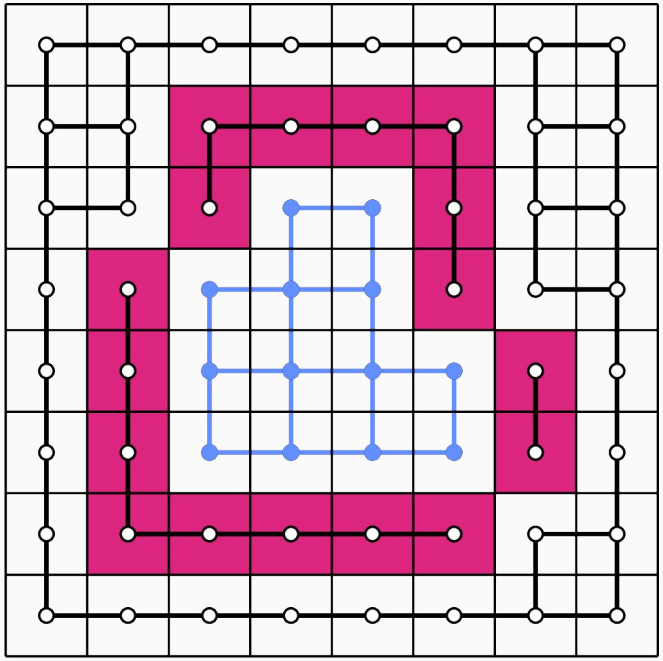


“Fill” tool in Paint

“Pretend” like you have a graph.

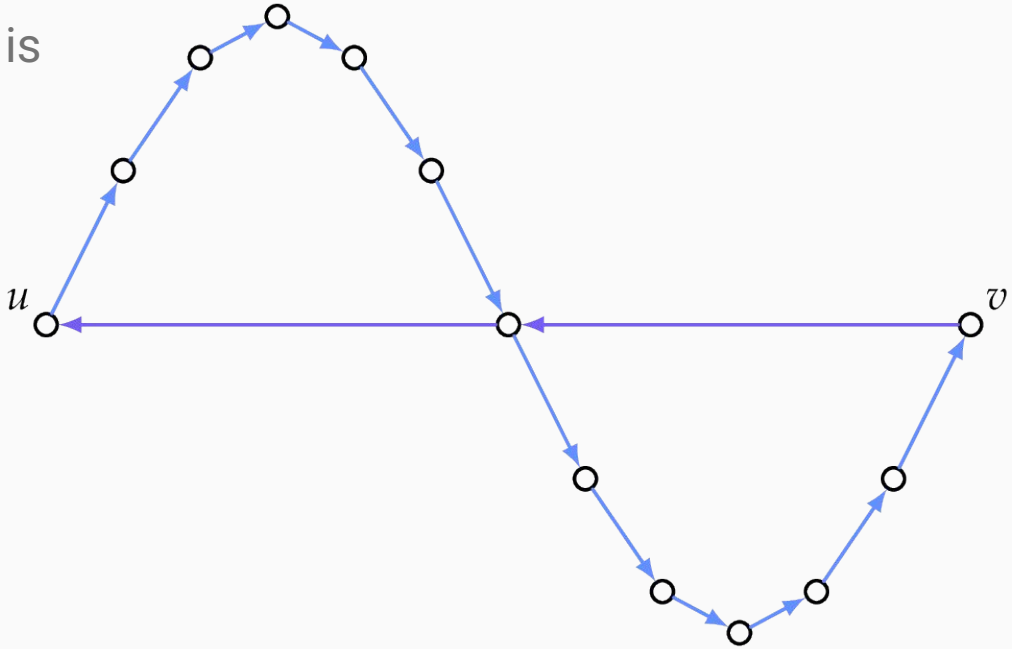
- Vertices are pixels
- Edge if neighbors are same color

The ***component*** of the selected pixel is colored.



Strong connectivity

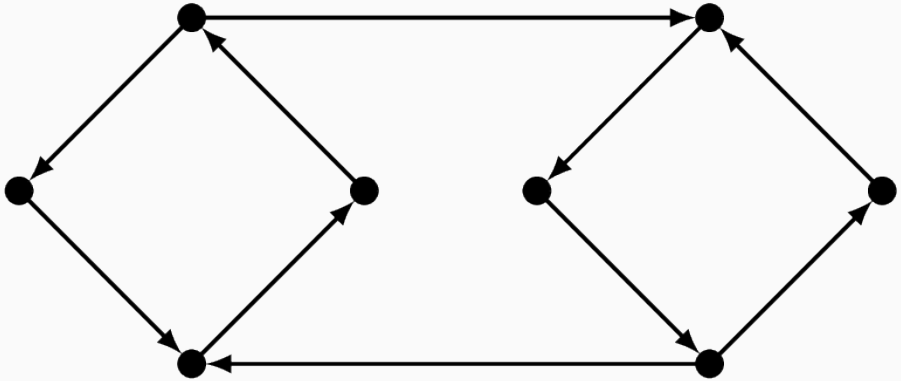
u and ***v*** are ***strongly connected*** if ***u*** is reachable from ***v***, and vice versa.



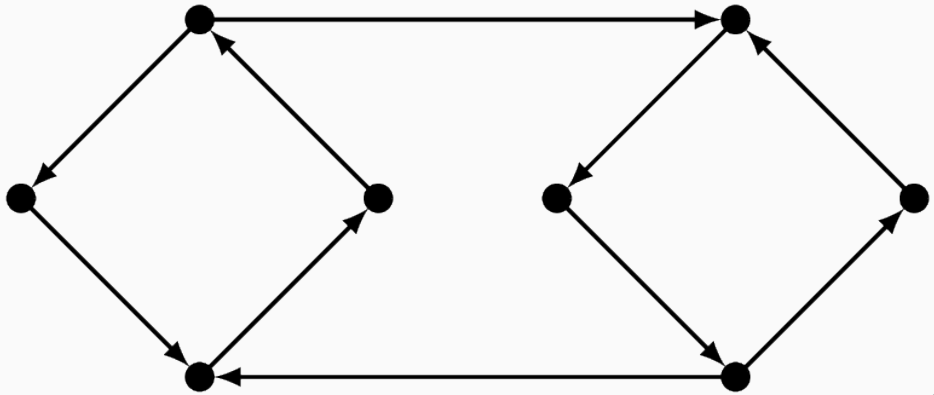
Strong connectivity

u and ***v*** are ***strongly connected*** if ***u*** is reachable from ***v***, and vice versa.

A graph ***G*** is ***strongly connected*** if every pair of vertices is strongly connected.

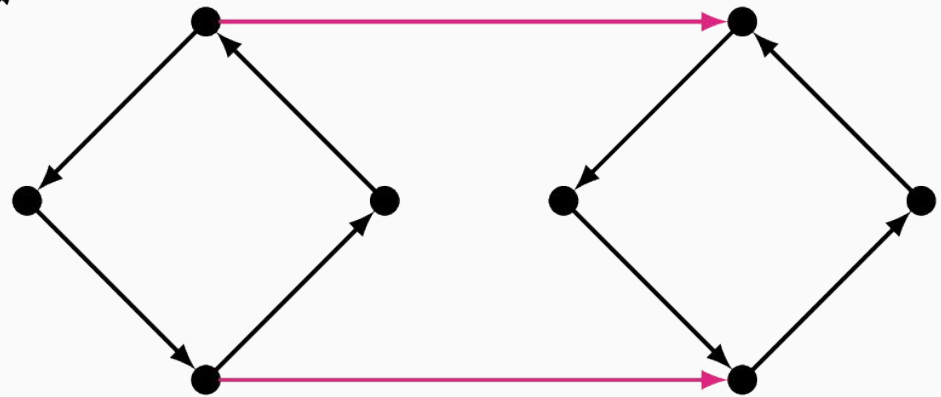


Strong connectivity



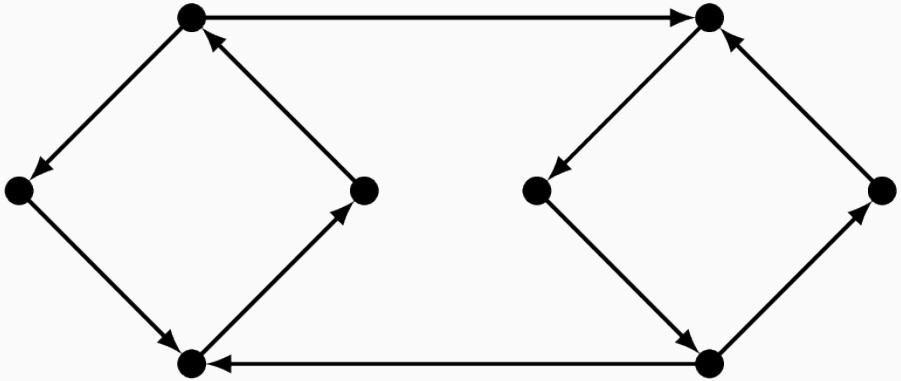
strongly connected

not strongly connected



Strong connectivity

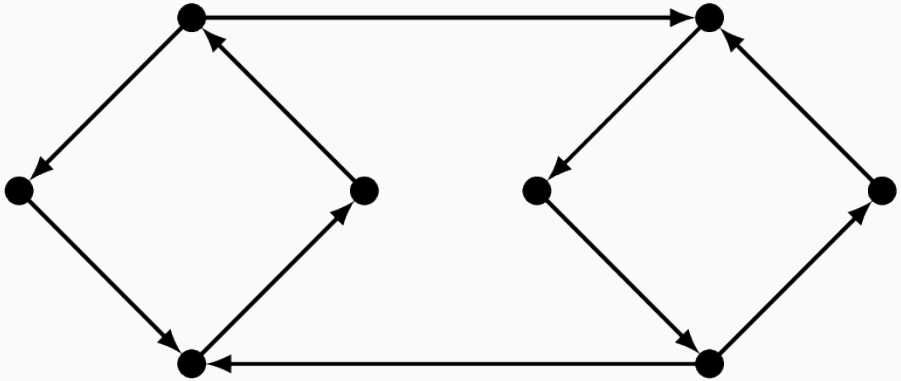
How can we check for strong connectivity?



Strong connectivity

How can we check for strong connectivity?

First attempt: do a DFS at every vertex and check if every vertex is reached

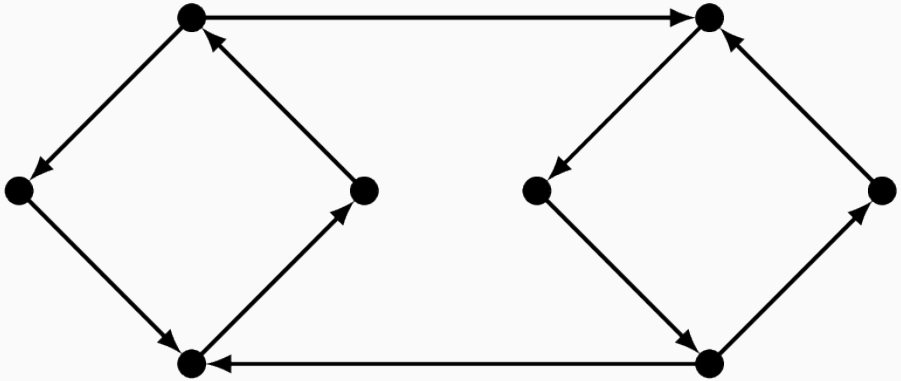


Strong connectivity

How can we check for strong connectivity?

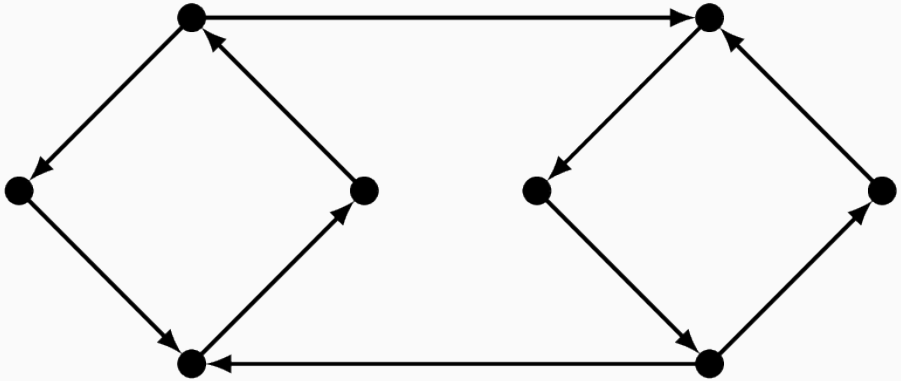
First attempt: do a DFS at every vertex and check if every vertex is reached

$O(|\boldsymbol{V}|(|\boldsymbol{V}|+|\boldsymbol{E}|))$ runtime



Strongly connected components

Strong connectivity is an ***equivalence relation*** on the vertices

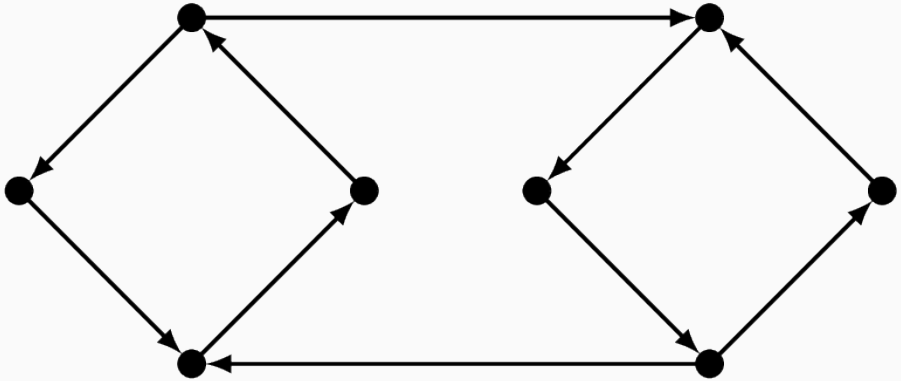


Strongly connected components

Strong connectivity is an ***equivalence relation*** on the vertices

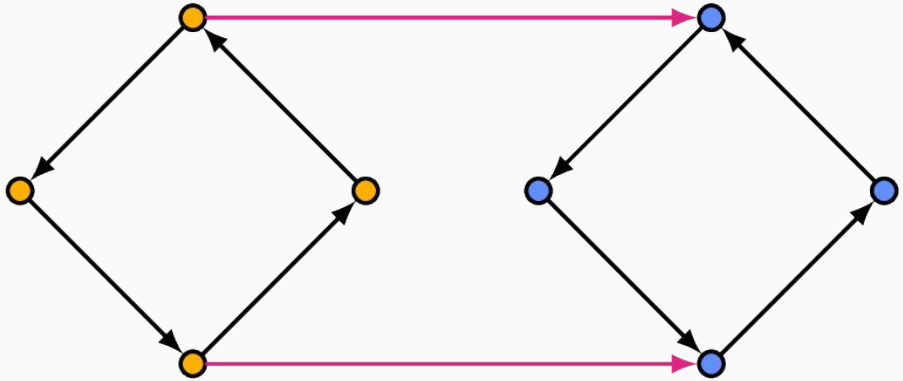
- Reflexive: $v \sim v$
- Symmetric: $u \sim v \rightarrow v \sim u$
- Transitive: $u \sim v$ and $v \sim w \rightarrow$

  $u \sim w$



Strongly connected components

Equivalence relations induce a ***partition*** into ***strongly connected components***.

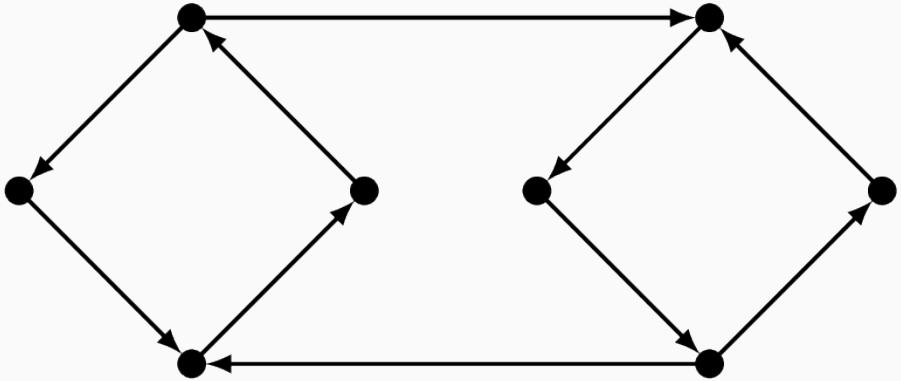


Strongly connected components

Second attempt:

1. Pick any ***v***
2. DFS(***v***) on forward edges
3. DFS(***v***) on backward edges

and check if every vertex is reached



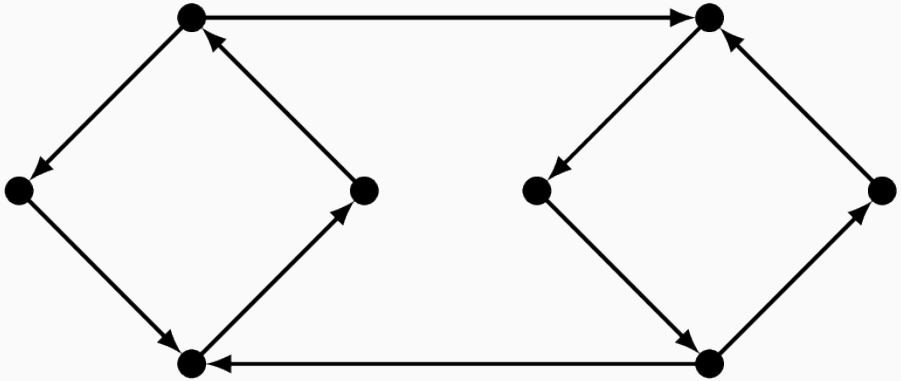
Strongly connected components

Second attempt:

1. Pick any ***v***
2. DFS(***v***) on forward edges
3. DFS(***v***) on backward edges

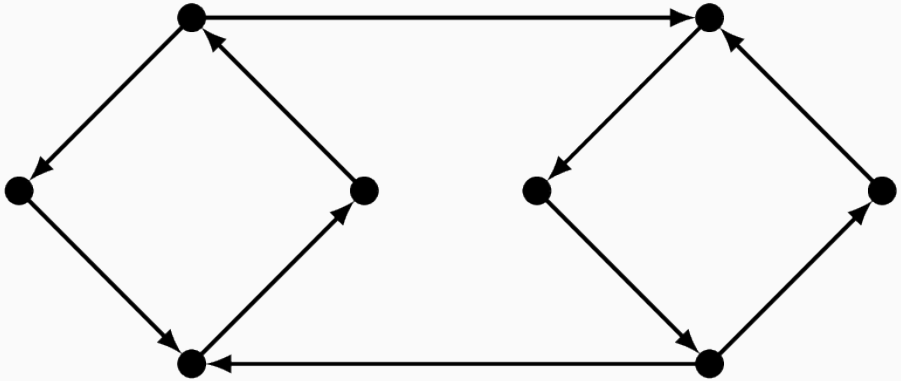
and check if every vertex is reached

Runtime: O(|***V***|+|***E***|), but why does it work?

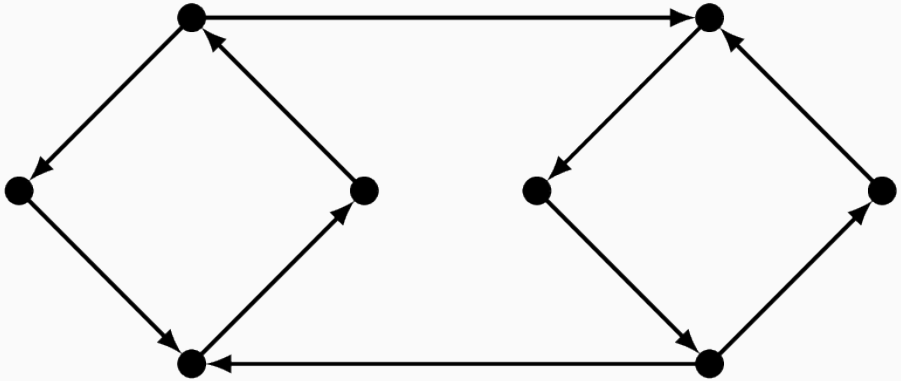


Strongly connected components

If the algorithm succeeds, every vertex belongs to the same SCC as ***v***.



Strongly connected components

If the algorithm succeeds, every vertex belongs to the same SCC as ***v***.

There's only one strongly connected component.